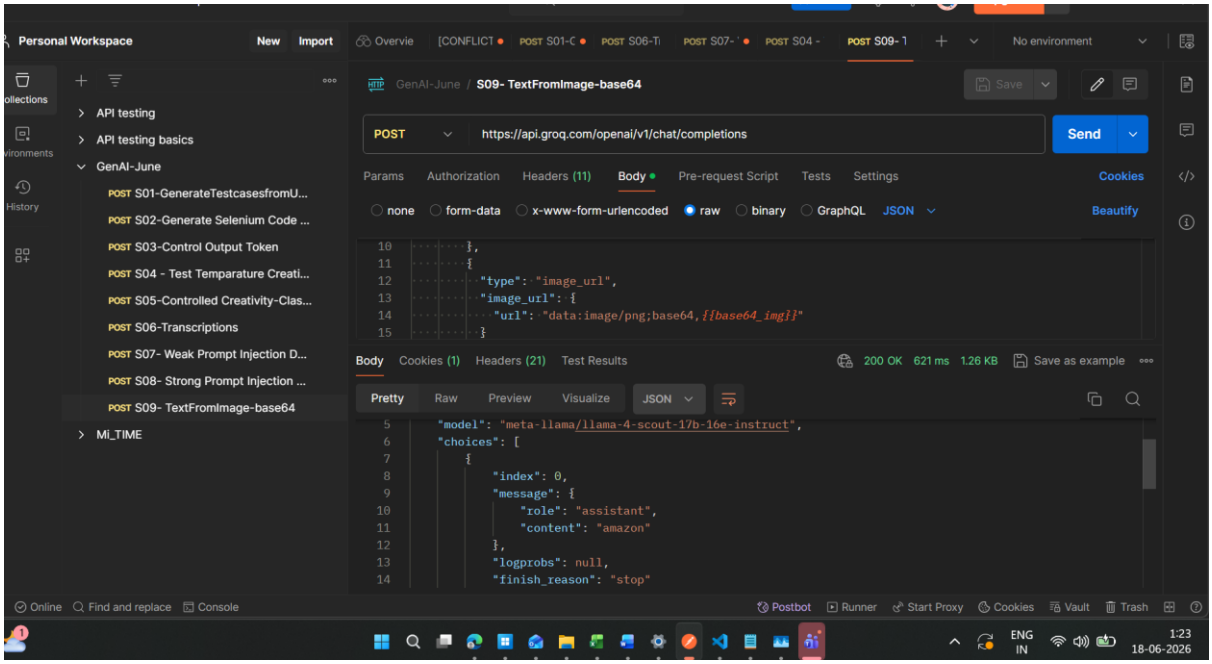
Week 2 – DAY 2 Assignments

Image to Text Converter

Image -

Converted –



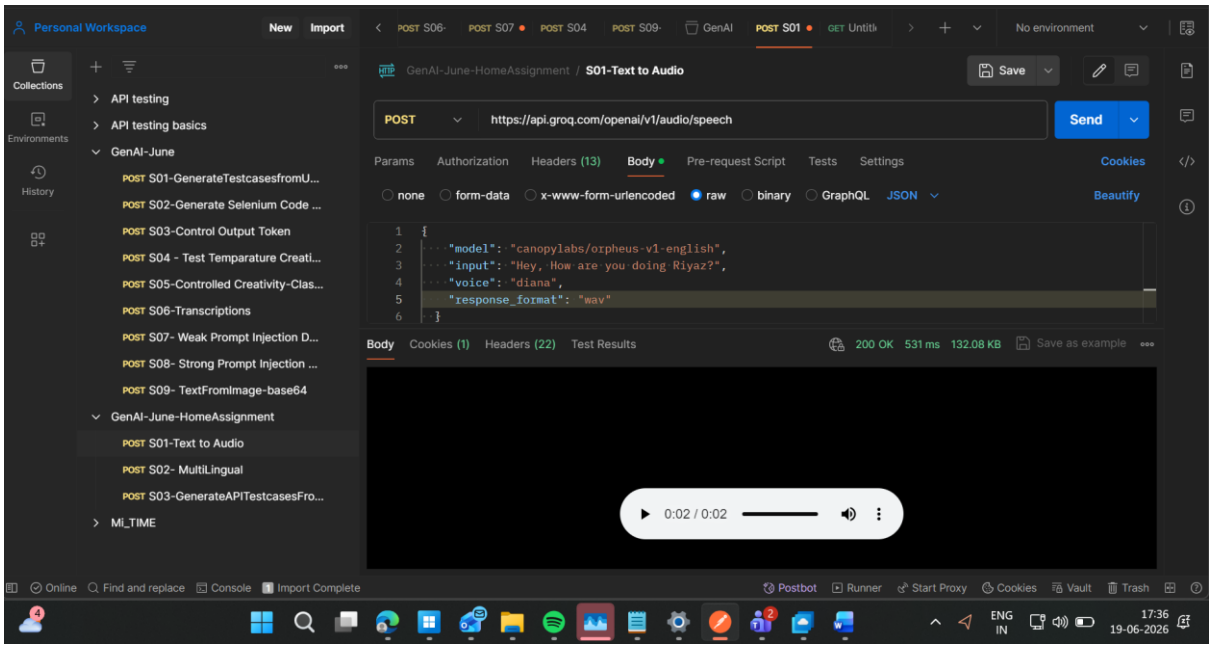
Sample response -


```
{
    "id": "chatcmpl-09a9fa49-8b58-4667-8b69-2723dd42a411",
    "object": "chat.completion",
    "created": 1781726017,
    "model": "meta-llama/llama-4-scout-17b-16e-instruct",
    "choices": [
        {
            "index": 0,
            "message": {
                "role": "assistant",
                "content": "amazon"
            },
            "logprobs": null,
            "finish_reason": "stop"
        }
    ],
    "usage": {
        "queue_time": 0.09495604,
        "prompt_tokens": 1917,
        "prompt_time": 0.083018416,
        "completion_tokens": 2,
        "completion_time": 0.004503988,
        "total_tokens": 1919,
        "total_time": 0.087522404
    },
    "usage_breakdown": null,
    "system_fingerprint": "fp_37da608fc1",
    "x_groq": {
```

```
        "id": "req_01kvbjaxgvfwmsags66knxbn7f",
        "seed": 1054311943
    },
    "service_tier": "on_demand"
}
```

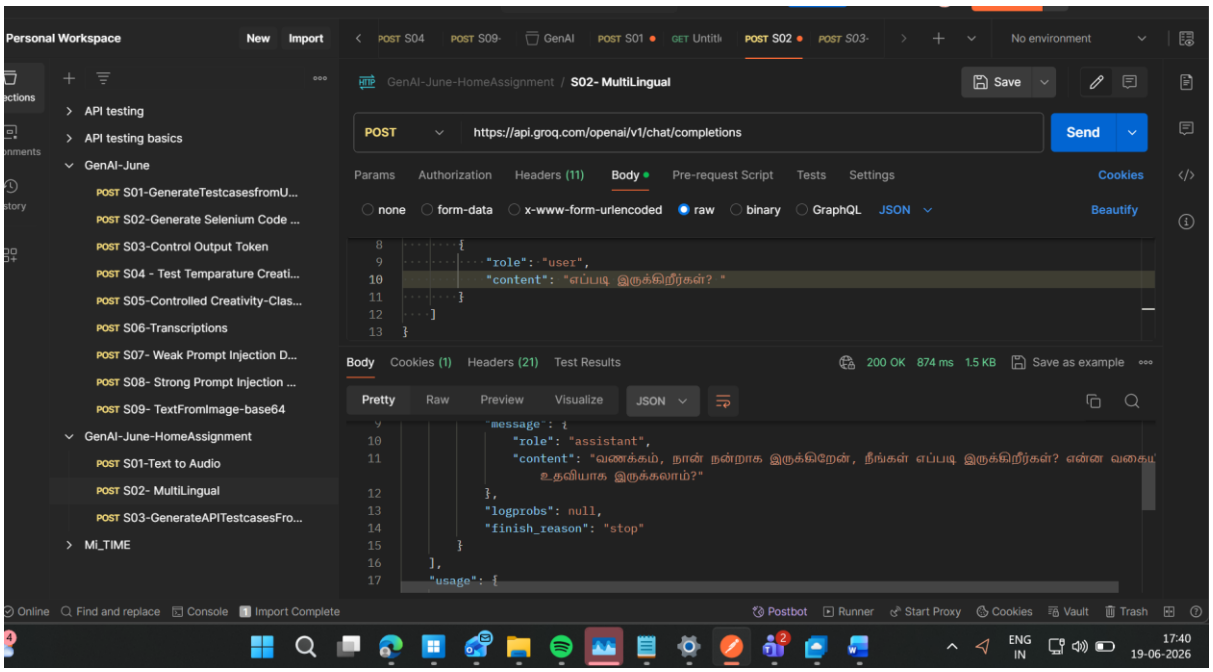

Home Assignments

Text to audio



Multi Lingual

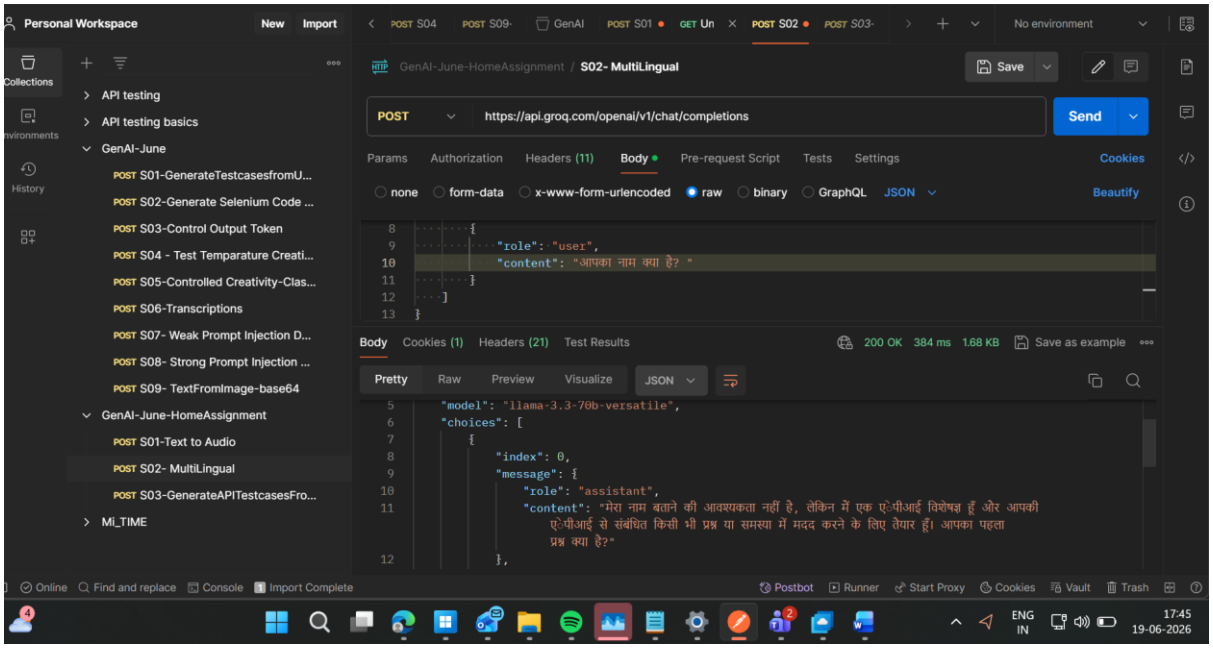
Example 1



Response -


```
{
   "id": "chatcmpl-d524df5b-502f-4e6b-be3c-937e04e2be37",
   "object": "chat.completion",
   "created": 1781871020,
   "model": "llama-3.3-70b-versatile",
   "choices": [
      {
         "index": 0,
         "message": {
            "role": "assistant",
            "content": "வணக்கம், நான் நன்றாக இருக்கிறேன், நீங்கள் எப்படி இருக்கிறீர்கள்? என்ன வகையில் உதவியாக இருக்கலாம்?"
         },
         "logprobs": null,
         "finish_reason": "stop"
      }
   ],
   "usage": {
      "queue_time": 0.464828171,
      "prompt_tokens": 82,
      "prompt_time": 0.016292908,
      "completion_tokens": 134,
      "completion_time": 0.361737877,
      "total_tokens": 216,
      "total_time": 0.378030785
   },
   "usage_breakdown": null,
   "system_fingerprint": "fp_0761e44d7b",
   "x_groq": {
      "id": "req_01kvfwm1t0ejqt7aqme80b92hh",
      "seed": 525141470
   },
   "service_tier": "on_demand"
}
```


Example 2-

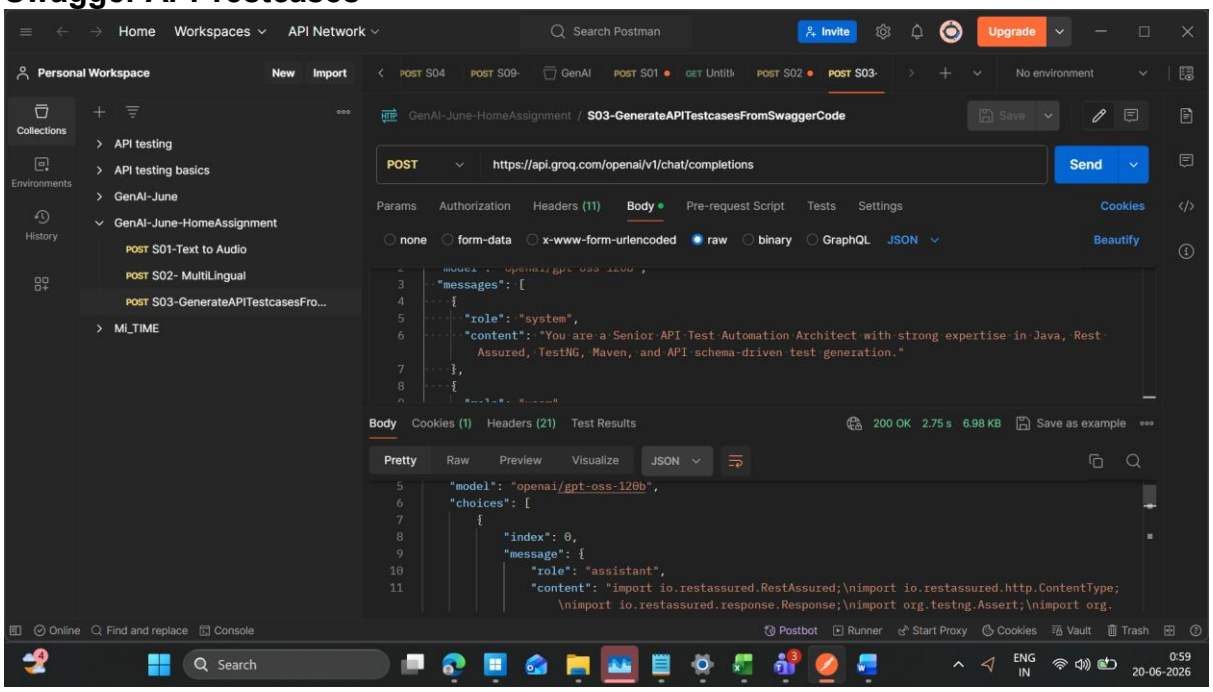


Response -


```
{
    "id": "chatcmpl-6f9a4068-22a3-452e-afcc-96141a7a2207",
    "object": "chat.completion",
    "created": 1781871202,
    "model": "llama-3.3-70b-versatile",
    "choices": [
        {
            "index": 0,
            "message": {
                "role": "assistant",
                "content": "मेरा नाम बताने की आवश्यकता नहीं है, लेकिन मैं एक एपीआई विशेषज्ञ हूँ और आपकी एपीआई से संबंधित किसी भी प्रश्न या समस्या में मदद करने के लिए तैयार हूँ। आपका पहला प्रश्न क्या है?"
            },
            "logprobs": null,
            "finish_reason": "stop"
        }
    ],
    "usage": {
        "queue_time": 0.050244026,
        "prompt_tokens": 62,
        "prompt_time": 0.002943304,
        "completion_tokens": 86,
        "completion_time": 0.304062959,
        "total_tokens": 148,
        "total_time": 0.307006263
    },
    "usage_breakdown": null,
    "system_fingerprint": "fp_dae98b5ecb",
    "x_groq": {
```

```
        "id": "req_01kvfwsm5qfdxabkzayrbf7ndf",
        "seed": 1064234090
    },
    "service_tier": "on_demand"
}
```

Swagger API Testcases



Sample Response -

```
import io.restassured.http.ContentType;
    import io.restassured.response.Response;
    import org.testng.Assert;
    import org.testng.annotations.BeforeClass;
    import org.testng.annotations.Test;

    import java.util.*;

    import static io.restassured.RestAssured.given;

    public class PetStoreCrudTests {

        private static final String BASE_PATH = "/v2";
        private static long petId;

        @BeforeClass
        public void setUp() {
            RestAssured.baseURI = "https://petstore.swagger.io";
            RestAssured.basePath = BASE_PATH;
            petId = new Random().nextInt(1_000_000);
        }
```

```
    @Test
    public void createPet() {
        Map<String, Object> pet = new HashMap<>();
        pet.put("id", petId);
        pet.put("name", "TestPet");
        pet.put("status", "available");

        Map<String, Object> category = new HashMap<>();
        category.put("id", 0);
        category.put("name", "Dogs");
        pet.put("category", category);

        pet.put("photoUrls",
Collections.singletonList("http://example.com/photo.jpg"));

        Map<String, Object> tag = new HashMap<>();
        tag.put("id", 0);
        tag.put("name", "tag1");
        pet.put("tags", Collections.singletonList(tag));

        Response response = given()
                .contentType(ContentType.JSON)
                .body(pet)
                .when()
                .post("/pet")
                .then()
                .extract()
                .response();

        Assert.assertEquals(response.getStatusCode(), 200);
        Assert.assertEquals(response.jsonPath().getLong("id"), petId);
        Assert.assertEquals(response.jsonPath().getString("name"), "TestPet");
        Assert.assertEquals(response.jsonPath().getString("status"), "available");
    }

    @Test(dependsOnMethods = "createPet")
    public void getPetById() {
        Response response = given()
                .accept(ContentType.JSON)
                .when()
                .get("/pet/{petId}", petId)
                .then()
                .extract()
                .response();

        Assert.assertEquals(response.getStatusCode(), 200);
        Assert.assertEquals(response.jsonPath().getLong("id"), petId);
        Assert.assertEquals(response.jsonPath().getString("name"), "TestPet");
        Assert.assertEquals(response.jsonPath().getString("status"), "available");
```

```
    }

    @Test(dependsOnMethods = "getPetById")
    public void updatePet() {
        Map<String, Object> updatedPet = new HashMap<>();
        updatedPet.put("id", petId);
        updatedPet.put("name", "UpdatedPet");
        updatedPet.put("status", "sold");

        Map<String, Object> category = new HashMap<>();
        category.put("id", 0);
        category.put("name", "Cats");
        updatedPet.put("category", category);

        updatedPet.put("photoUrls",
Collections.singletonList("http://example.com/updated.jpg"));

        Map<String, Object> tag = new HashMap<>();
        tag.put("id", 1);
        tag.put("name", "tag2");
        updatedPet.put("tags", Collections.singletonList(tag));

        Response response = given()
                .contentType(ContentType.JSON)
                .body(updatedPet)
                .when()
                .put("/pet")
                .then()
                .extract()
                .response();

        Assert.assertEquals(response.getStatusCode(), 200);
        Assert.assertEquals(response.jsonPath().getLong("id"), petId);
        Assert.assertEquals(response.jsonPath().getString("name"), "UpdatedPet");
        Assert.assertEquals(response.jsonPath().getString("status"), "sold");
    }

    @Test(dependsOnMethods = "updatePet")
    public void deletePet() {
        Response response = given()
                .when()
                .delete("/pet/{petId}", petId)
                .then()
                .extract()
                .response();

        Assert.assertEquals(response.getStatusCode(), 200);
        Assert.assertTrue(response.asString().contains(String.valueOf(petId)));
    }
}
```

reasoning: We need to output Java code only, no markdown fences. Include imports, class, @BeforeClass, test methods for createPet, getPetById, updatePet, deletePet. Use Rest Assured, TestNG, assertions. Reusable base URI. Use meaningful names. Must cover CRUD. Provide request body creation (likely using a POJO or Map). Could use JSONObject or HashMap. Use static petId variable to share across tests. Ensure test execution order? TestNG doesn't guarantee order unless dependsOnMethods. We'll use dependsOnMethods for sequential.

We need to include imports: io.restassured.RestAssured, io.restassured.http.ContentType, io.restassured.response.Response, org.testng.annotations. , org.testng.Assert.*

Set baseURI in @BeforeClass: RestAssured.baseURI = "https://petstore.swagger.io/v2";

Create pet JSON body: id, name, status, maybe category, photoUrls, tags.

We'll generate random id using java.util.UUID? Use long. For simplicity, static long petId = new Random().nextInt(1000000); We'll import java.util.Random.

We'll create methods: createPet, getPetById, updatePet, deletePet. Use dependsOnMethods.

Assertions: status code 200 for create, get, update, delete (delete returns 200). Validate response fields: id matches, name, status.

We'll use Response response = given().contentType(ContentType.JSON).body(petJson).when().post("/pet").

Construct petJson as a Map<String,Object>.

Ok produce code.